

# Parametric Quantum Circuit Bases for Image Compression via Riemannian Optimization on Unitary Manifolds

Author One<sup>1</sup>

2026-03-18

We present a unified framework for learning data-adaptive unitary transforms by relaxing the gate parameters of quantum circuit tensor networks and optimizing them on Riemannian manifolds. The Hadamard gates are optimized on the unitary group  $U(2)$  via Cayley retraction, while controlled-phase gates are optimized on the product of unit circles  $U(1)^4$ , preserving unitarity exactly at every iterate. We systematically compare four circuit topologies—separable Quantum Fourier Transform (QFT), an Entangled QFT with cross-dimensional coupling, Time-Evolving Block Decimation (TEBD) with ring connectivity, and a MERA-inspired hierarchical circuit—on image compression benchmarks including Quick Draw ( $32 \times 32$ ) and DIV2K ( $256 \times 256$ ). The Entangled QFT, which introduces  $n$  additional phase gates coupling row and column qubits, achieves the best compression quality across all settings, while TEBD with local ring connectivity underperforms the fixed classical FFT by up to 1.5 dB despite having comparable parameter count. The results indicate that circuit connectivity topology is a more important design variable than parameter count or circuit depth for learned transform bases. The approach brings together concepts from tensor networks, Riemannian optimization, quantum circuit simulation, and differentiable programming, and is implemented in an open-source Julia library with GPU acceleration.

## 1 Introduction

Sparse representation of signals is fundamental to data compression. Classical transforms such as the Discrete Fourier Transform (DFT) [1] and the Discrete Cosine Transform (DCT) [2] provide fixed bases with well-understood sparsity properties. However, these bases are not adaptive to the statistics of specific signal classes. In particular, the separable 2D DFT  $F_m \otimes F_n$  treats spatial dimensions independently, ignoring cross-dimensional correlations common in natural images.

The Cooley–Tukey FFT algorithm admits a natural decomposition as a tensor network of two-qubit gates: Hadamard gates  $H$  and controlled-phase gates  $M_k = \text{diag}(1, 1, 1, e^{i\pi/2^{k-1}})$ . With fixed parameters, this tensor network computes the DFT exactly. By relaxing the gate parameters—allowing  $H$  to range over the full unitary group  $U(2)$  and the phases of  $M_k$  to become learnable—we obtain a continuous family of parametric unitary transforms that includes the DFT as a special case. The optimization seeks the point on this parameter manifold that yields the best compression quality for a given dataset.

In this paper, we present a unified framework for learning such parametric quantum circuit bases and systematically compare four circuit topologies: the separable QFT, an Entangled QFT with cross-dimensional coupling, TEBD [3] with ring connectivity, and a MERA-inspired [4] hierarchical circuit. The circuit parameters are optimized on Riemannian manifolds ( $U(2)$  for Hadamard gates,  $U(1)^4$  for controlled-phase gates) using Cayley retractions and Riemannian Adam with parallel transport [5], preserving unitarity exactly at every iterate. We employ a frequency-weighted top- $k$  truncation with a straight-through gradient estimator to enable

end-to-end differentiable training of the reconstruction loss.

Our experiments on Quick Draw ( $32 \times 32$ ) and DIV2K ( $256 \times 256$ ) benchmarks reveal that circuit connectivity topology is a more important design variable than parameter count or circuit depth. The Entangled QFT, which adds only  $n$  cross-dimensional phase gates to the separable QFT, consistently achieves the best compression quality, while TEBD with comparable parameter count but local ring connectivity underperforms even the fixed FFT.

The approach brings together techniques from tensor networks, Riemannian optimization, quantum circuit simulation [6], and differentiable programming into a unified computational framework. The implementation is available as an open-source Julia library `ParametricDFT.jl` with batched einsum contractions via `OMEinsum.jl` [7], Riemannian optimizers, and optional GPU acceleration, building on the Julia ecosystem for quantum computation and tensor network methods.

**Related work.** Tensor network methods have been applied to machine learning tasks including classification [8], generative modeling [9], and data compression [10]. Parameterized quantum circuits underpin variational quantum algorithms [11, 12] and quantum machine learning [13, 14]. Riemannian optimization for unitary constraints appears in quantum control [15] and optimization on matrix manifolds [16]. Tropical tensor networks [17] and generic tensor networks [18] have demonstrated the power of combining tensor network contraction with automatic differentiation and GPU acceleration for combinatorial optimization. Our work applies a similar synthesis—tensor networks, differentiable programming, and manifold optimization—to the problem of learning sparse transform bases for image compression.

**Outline.** Section 2 reviews the tensor network representation of the FFT. Section 3 presents the four circuit topologies. Section 4 describes the loss functions, truncation scheme, and Riemannian optimizers. Section 5 reports benchmark results. Section 6 discusses implications and future directions.

## 2 Background: Tensor Network Representation of the FFT

We briefly review how the Cooley–Tukey FFT decomposes into a tensor network, establishing notation used throughout.

### 2.1 Cooley–Tukey decomposition

The DFT matrix of size  $n = 2^k$  is  $(F_n)_{ij} = \omega^{(i-1)(j-1)}$  where  $\omega = e^{-2\pi i/n}$ . The Cooley–Tukey algorithm decomposes this as

$$F_n \mathbf{x} = \begin{pmatrix} I_{n/2} & D_{n/2} \\ I_{n/2} & -D_{n/2} \end{pmatrix} \begin{pmatrix} F_{n/2} & 0 \\ 0 & F_{n/2} \end{pmatrix} \begin{pmatrix} \mathbf{x}_{\text{odd}} \\ \mathbf{x}_{\text{even}} \end{pmatrix}, \quad (1)$$

where  $D_{n/2} = \text{diag}(1, \omega, \omega^2, \dots, \omega^{n/2-1})$  is the twiddle factor matrix.

### 2.2 Gate decomposition

In tensor network notation, a vector of size  $2^k$  is a rank- $k$  tensor with binary indices  $q_0, q_1, \dots, q_{k-1}$ . The butterfly structure of eq. (1) decomposes into:

- **Hadamard gates:**  $H = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ , one per qubit.
- **Controlled-phase gates:**  $M_k = \text{diag}(1, 1, 1, e^{i\pi/2^{k-1}})$ , connecting each pair of qubits at distance  $k$  in the recursive decomposition.

Recursive application yields  $k$  Hadamard gates and  $k(k-1)/2$  controlled-phase gates for  $k$  qubits, evaluable in  $O(2^k \cdot k)$  operations via optimized tensor contraction—matching the classical FFT complexity  $O(N \log N)$  for  $N = 2^k$ .

For 2D images of size  $2^m \times 2^n$ , the separable 2D DFT is  $F_m \otimes F_n$ , requiring  $m + n$  Hadamard gates and  $m(m-1)/2 + n(n-1)/2$  controlled-phase gates applied independently to the row and column qubit registers.

## 3 Parametric Circuit Topologies

We now describe four circuit topologies, each defining a parametric unitary transform  $\mathcal{T}(\theta) : \mathbb{C}^{2^m \times 2^n} \rightarrow \mathbb{C}^{2^m \times 2^n}$ . All topologies share the same gate parameterization: Hadamard-like gates  $H \in U(2)$  and controlled-phase gates  $M =$

$\text{diag}(1, 1, 1, e^{i\phi}) \in U(1)^4$  with learnable parameters. Setting all parameters to their QFT values recovers the classical DFT.

### 3.1 QFT basis

The QFT basis directly parameterizes the tensor network of section 2.2. Each gate becomes a learnable parameter: the  $m + n$  Hadamard gates live on  $U(2)$  and the  $m(m - 1)/2 + n(n - 1)/2$  controlled-phase gates live on  $U(1)^4$ . The row and column circuits operate independently, yielding the separable transform  $\mathcal{T} = F_m(\boldsymbol{\theta}_{\text{row}}) \otimes F_n(\boldsymbol{\theta}_{\text{col}})$ .

**Parameter count.** For an image of size  $2^m \times 2^n$ :  $(m + n)$  unitary gates on  $U(2)$  (4 real parameters each) plus  $m(m - 1)/2 + n(n - 1)/2$  phase gates on  $U(1)^4$  (1 real parameter each), giving  $4(m + n) + m(m - 1)/2 + n(n - 1)/2$  total real parameters.

### 3.2 Entangled QFT basis

The separable QFT processes row and column dimensions independently, ignoring spatial correlations between them. Natural images exhibit strong cross-dimensional correlations (e.g., diagonal edges, radial textures). We propose an *Entangled QFT* that introduces controlled-phase gates coupling corresponding row and column qubits.

For a square image with  $n$  qubits per dimension, we add  $n$  entanglement gates  $E_k = \text{diag}(1, 1, 1, e^{i\phi_k})$  acting on qubit pairs  $(x_{n-k}, y_{n-k})$  for  $k = 1, \dots, n$ . Each gate multiplies a learnable phase  $e^{i\phi_k}$  when both the row qubit  $x_{n-k}$  and column qubit  $y_{n-k}$  are in state  $|1\rangle$ .

The total transformation is:

$$\mathcal{T}_{\text{entangled}} = U_{\text{entangle}} \cdot (F_n \otimes F_n), \quad U_{\text{entangle}} = \prod_{k=1}^n E_k. \quad (2)$$

This extension has several properties:

1. Computational complexity remains  $O(N \log N)$  per dimension.
2. Only  $n$  additional real parameters are introduced ( $O(\log N)$  overhead).
3. Setting  $\phi_k = 0$  for all  $k$  recovers the separable QFT.

### 3.3 TEBD basis

Time-Evolving Block Decimation (TEBD) [3] uses a *ring topology* of nearest-neighbor controlled-phase gates preceded by Hadamard gates on each qubit. For  $n$  row qubits and  $n$  column qubits, each register forms an independent ring: qubit  $q_i$  is connected to  $q_{i+1}$  (with wrap-around  $q_n \rightarrow q_1$ ).

Each gate is a controlled-phase gate  $T_k = \text{diag}(1, 1, 1, e^{i\phi_k})$  with a single learnable phase. The row ring has  $n$  gates and the column ring has  $n$  gates, giving  $2n$  total learnable phases. The TEBD circuit evaluates in  $O(n)$  gate operations per dimension, making it the computationally cheapest basis at inference time.

The parameter manifold is:

$$\mathcal{M}_{\text{TEBD}} = \prod_{k=1}^{2n} U(1)^4. \quad (3)$$

### 3.4 MERA-inspired basis

The Multi-scale Entanglement Renormalization Ansatz (MERA) [4] uses hierarchical connectivity with *disentangler*s and *isometries* at increasing spatial scales. Our MERA-inspired basis borrows MERA's connectivity pattern while using controlled-phase gates (rather than full  $U(4)$  unitaries) for consistency with the other bases and to preserve invertibility.

For  $n = 2^k$  qubits, the circuit has  $k = \log_2 n$  layers. Layer  $l$  has stride  $s = 2^{l-1}$  and contains  $n/(2s)$  pairs of disentangler and isometry gates. Both gate types use the same parameterization:  $\text{diag}(1, 1, 1, e^{i\phi})$  with one learnable phase per gate.

The total gate count per dimension is  $2(n - 1)$ , and the circuit depth is  $O(\log n)$ . The hierarchical connectivity captures multi-scale features: layer 1 acts on nearest neighbors (fine scale), while deeper layers connect distant qubits (coarse scale).

### 3.5 Summary of circuit properties

## 4 Training Objective and Optimization

### 4.1 Loss functions

We consider three loss functions with different trade-offs between sparsity promotion and reconstruction quality.

Table 1: Comparison of the four circuit topologies for an image of size  $2^n \times 2^n$  ( $n$  qubits per dimension).

| Property      | QFT        | Ent. QFT        | TEBD   | MEBA          |
|---------------|------------|-----------------|--------|---------------|
| Connectivity  | all-to-all | all-to-all + XY | ring   | hierarchical  |
| Phase params  | $n(n-1)$   | $n(n-1) + n$    | $2n$   | $4(n-1)$      |
| $U(2)$ gates  | $2n$       | $2n$            | $2n$   | $2n$          |
| Circuit depth | $O(n^2)$   | $O(n^2)$        | $O(n)$ | $O(n \log n)$ |
| XY coupling   | No         | Yes             | No     | No            |

**$L_1$  norm loss.**  $\mathcal{L}_{L_1}(\theta) = \sum_{i,j} |\mathcal{T}(\theta)(\mathbf{x})_{i,j}|$ . The  $\ell_1$  norm is the tightest convex relaxation of the  $\ell_0$  pseudo-norm, promoting sparse representations.

**$L_2$  norm loss.**  $\mathcal{L}_{L_2}(\theta) = \sum_{i,j} |\mathcal{T}(\theta)(\mathbf{x})_{i,j}|^2$ . Encourages energy concentration with smoother gradients than  $L_1$ .

**MSE reconstruction loss.**

$$\mathcal{L}_{\text{MSE}}(\theta) = \|\mathbf{x} - \mathcal{T}(\theta)^{-1}(\text{topk}(\mathcal{T}(\theta)(\mathbf{x}), k))\|_F^2, \quad (4)$$

where  $\text{topk}(\cdot, k)$  retains the  $k$  coefficients with the highest frequency-weighted scores and zeros out the rest. This loss directly optimizes reconstruction quality after truncation and is used in all experiments.

## 4.2 Frequency-weighted truncation

The top- $k$  selection uses a frequency-weighted score rather than raw magnitude:

$$s_{i,j} = |y_{i,j}| \cdot (1 + w_{i,j}), \quad w_{i,j} = 1 - \frac{d_{i,j}}{2d_{\max}}, \quad (5)$$

where  $d_{i,j} = \sqrt{(i - c_i)^2 + (j - c_j)^2}$  is the distance from frequency bin  $(i, j)$  to the DC component and  $d_{\max}$  is the maximum such distance. Low-frequency coefficients receive up to a  $2\times$  boost in their retention score, consistent with the human visual system’s greater sensitivity to low-frequency content.

## 4.3 Straight-through gradient for top- $k$

The top- $k$  truncation is non-differentiable: the set of retained indices changes discontinuously. We define a custom chain rule that passes gradients through retained coefficients and zeros out

the rest—the *straight-through estimator* applied to truncation:

$$\frac{\partial \text{topk}(\mathbf{y}, k)}{\partial y_{i,j}} = \begin{cases} 1 & \text{if } (i, j) \in \mathcal{S}, \\ 0 & \text{otherwise,} \end{cases} \quad (6)$$

where  $\mathcal{S}$  is the set of retained indices. This is justified because (i) the true Jacobian has delta-function contributions at the boundaries that are unusable for gradient-based optimization, (ii) the mask is locally stable during training, and (iii) the gradient correctly conveys “adjust the retained coefficients to better approximate the signal.”

## 4.4 Riemannian optimization

The circuit parameters live on two manifolds:

**Definition 1** (Unitary manifold  $U(2)$ ). *The group of  $2 \times 2$  unitary matrices with tangent space  $T_U U(2) = \{US : S^\dagger = -S\}$ .*

**Definition 2** (Phase manifold  $U(1)^4$ ). *The product of four unit circles with tangent space  $T_{\mathbf{z}} U(1)^4 = \{i\theta \circ \mathbf{z} : \theta \in \mathbb{R}^4\}$ .*

**Riemannian gradient.** Given the Euclidean gradient  $\nabla_E f$  from automatic differentiation:

$$\text{For } U(2): \quad \text{grad } f(U) = U \cdot \text{skew}(U^\dagger \nabla_E f), \quad (7)$$

$$\text{For } U(1)^4: \quad \text{grad } f(\mathbf{z}) = i \cdot \text{Im}(\bar{\mathbf{z}} \circ \nabla_E f) \circ \mathbf{z}, \quad (8)$$

where  $\text{skew}(A) = (A - A^\dagger)/2$  extracts the skew-Hermitian part.

**Retraction.** For  $U(2)$ , we use the Cayley retraction:

$$R_U(\alpha\xi) = \left(I - \frac{\alpha}{2}W\right)^{-1} \left(I + \frac{\alpha}{2}W\right)U, \quad (9)$$

where  $W = \xi U^\dagger$  projected to  $\mathfrak{u}(2)$  via  $W \leftarrow (W - W^\dagger)/2$ . The Cayley map preserves unitarity exactly for any step size  $\alpha$ . For  $U(1)^4$ , we use normalization:  $R_{\mathbf{z}}(\alpha\xi) = (\mathbf{z} + \alpha\xi)/|\mathbf{z} + \alpha\xi|$  elementwise.

**Riemannian gradient descent with Armijo line search.** The update rule is  $x_{t+1} = R_{x_t}(-\eta \cdot \text{grad } f(x_t))$ , where  $\eta$  is determined by Armijo backtracking: starting from  $\alpha_0$ , contract  $\alpha \leftarrow \tau \alpha$  (with  $\tau = 0.5$ ) until the sufficient decrease condition is satisfied:

$$\mathcal{L}(R_{x_t}(-\alpha \cdot \text{grad } f(x_t))) \leq \mathcal{L}(x_t) - c \cdot \alpha \cdot \|\text{grad } f(x_t)\|^2, \quad (10)$$

with  $c = 10^{-4}$ .

**Riemannian Adam.** We also implement Riemannian Adam [5], which maintains exponentially weighted first and second moment estimates of the Riemannian gradient with parallel transport of momentum between iterates. The bias-corrected update direction is  $d_t = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ , with  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$ . Momentum is transported between tangent spaces via projection:

$$\Gamma_{x_t \rightarrow x_{t+1}}(m_t) = \text{proj}_{T_{x_{t+1}}} \mathcal{M}(m_t). \quad (11)$$

#### 4.5 Batched einsum evaluation

The forward transform is implemented as an Einstein summation (einsum) contraction of the image tensor with the gate tensors. The contraction order is pre-optimized using a tree search algorithm (TreeSA) that minimizes the total operation count.

For batched training, a batch index  $\beta$  is appended to the image input/output indices:

$$Y_{q'_1, \dots, q'_{m+n}, \beta} = \sum_{q_1, \dots, q_{m+n}} \left( \prod_j G_{\dots}^j \right) X_{q_1, \dots, q_{m+n}, \beta}. \quad (12)$$

The gate tensors are shared across the batch, reducing GPU kernel launch overhead from  $O(B)$  to  $O(1)$ .

## 5 Experiments

### 5.1 Setup

**Datasets.** We evaluate on two datasets of different scales:

- **Quick Draw** [19]: Hand-drawn sketches ( $28 \times 28$ , zero-padded to  $32 \times 32$ , i.e.,  $m = n = 5$  qubits). Five categories (cat, dog, airplane, apple, bicycle), 1000+ images per category.

- **DIV2K** [20]: High-resolution natural photographs, preprocessed to  $256 \times 256$  ( $m = n = 8$  qubits). 800 training + 100 validation images.

**Training configuration.** All bases are trained using RiemannianGD with Armijo line search ( $\alpha_0 = 0.01$ ,  $c = 10^{-4}$ ,  $\tau = 0.5$ , max 10 backtracking steps). The loss function is  $\mathcal{L}_{\text{MSE}}$  (eq. (4)) with  $k = 10\%$  of total coefficients. We use a 20% validation split, early stopping with patience 2 epochs, and random seed 42 for reproducibility.

**Metrics.** We report PSNR (peak signal-to-noise ratio,  $10 \log_{10}(1/\text{MSE})$ , higher is better) and SSIM (structural similarity index, higher is better) at compression ratios  $k/N \in \{5\%, 10\%, 15\%, 20\%\}$ . All metrics are averaged over the full evaluation set.

**Baselines.** We compare against the classical FFT (the untrained fixed-parameter DFT), which serves as the natural baseline since all learned bases reduce to it at their initialization point.

### 5.2 Results on Quick Draw ( $32 \times 32$ )

Table 2: Compression quality on Quick Draw ( $32 \times 32$ , 5 qubits per dimension). PSNR (dB) / SSIM at four compression ratios. Best learned result in **bold**.

| Method             | 5%                          | 10%                         |            |
|--------------------|-----------------------------|-----------------------------|------------|
| FFT (fixed)        | 15.27 / 0.363               | 17.27 / 0.444               | 18.        |
| QFT (learned)      | 15.78 / 0.364               | 17.42 / 0.444               | 18.        |
| Ent. QFT (learned) | <b>15.38</b> / <b>0.379</b> | <b>17.42</b> / <b>0.458</b> | <b>19.</b> |
| TEBD (learned)     | 14.57 / 0.340               | 16.35 / 0.414               | 17.        |

On Quick Draw (table 2), the Entangled QFT achieves the best performance across all compression ratios, improving over the classical FFT by +0.20 dB PSNR and +0.015 SSIM at 20% compression. The separable QFT shows modest gains at aggressive compression (5%) but converges to FFT performance at higher ratios, suggesting that the learned parameters remain close to the DFT values. TEBD consistently underperforms the fixed FFT baseline by 0.92 dB at 10% compression, indicating that the ring topology’s nearest-neighbor connectivity is insufficient



for the frequency structure of even simple sketch images.

### 5.3 Results on DIV2K ( $256 \times 256$ )

Table 3: Compression quality on DIV2K ( $256 \times 256$ , 8 qubits per dimension). PSNR (dB) / SSIM at four compression ratios. Best learned result in **bold**.

| Method             | 5%                          | 10%                         |
|--------------------|-----------------------------|-----------------------------|
| FFT (fixed)        | 27.43 / 0.728               | 29.76 / 0.805               |
| QFT (learned)      | 25.78 / 0.698               | 28.20 / 0.773               |
| Ent. QFT (learned) | <b>25.79</b> / <b>0.699</b> | <b>28.20</b> / <b>0.773</b> |
| TEBD (learned)     | 24.49 / 0.614               | 26.61 / 0.685               |

On DIV2K (table 3), all QFT-family bases improve over the FFT baseline, with the Entangled QFT and separable QFT performing comparably. The gain from entanglement is smaller on high-resolution images (+0.01 dB), likely because the 8-qubit QFT structure already provides sufficient all-to-all connectivity within each dimension. TEBD’s deficit grows from 0.92 dB on  $32 \times 32$  to 1.51 dB on  $256 \times 256$ , consistent with the expectation that the ring topology’s locality becomes more limiting as image complexity increases.

### 5.4 Training dynamics

Table 4: Training time (seconds) and convergence behavior on DIV2K.

| Method   | Time (s) | Final train loss | Final val loss |
|----------|----------|------------------|----------------|
| QFT      | 290.8    | 1162.0           | 2491.3         |
| Ent. QFT | 126.9    | 1162.0           | 2490.9         |
| TEBD     | 277.8    | 1752.6           | 1441.2         |

The Entangled QFT converges faster than the separable QFT (127 s vs. 291 s on DIV2K), despite having more parameters. We attribute this to the cross-dimensional gradient pathways created by the entanglement gates, which provide additional directions for the optimizer to explore. TEBD converges to a training loss approximately 50% higher than the QFT variants (1752.6 vs. 1162.0). Notably, TEBD’s validation loss is lower than its training loss, suggesting that the ring topology acts as an implicit regularizer at the cost of underfitting.

### 5.5 Effect of circuit topology

The key experimental finding is that **circuit connectivity topology determines compression quality more than parameter count or circuit depth**:

1. The Entangled QFT adds only  $n$  phase parameters over the QFT (+8 parameters for DIV2K) yet consistently achieves the best or tied-best performance.
2. TEBD/ 0.850 282.92 phase 880 parameters—comparable to the Entangled QFT—but performs worst due to its local ring connectivity.
3. The QFT’s all-to-all connectivity within each dimension closely matches the DFT’s frequency structure, explaining why learned QFT bases converge near the DFT solution.
4. Cross-dimensional coupling (Entangled QFT) captures correlations that no amount of intra-dimensional parameters can express in a separable architecture.

## 6 Discussion

The learned bases improve upon the classical FFT but do not dramatically exceed it. This is consistent with the DFT being near-optimal within the separable transform family for generic images [21]: the separable QFT’s learned parameters converge close to DFT values, confirming that the FFT occupies a strong local minimum on the parameter manifold. The Entangled QFT improves upon this by expanding the search space with cross-dimensional connections that the separable manifold cannot reach.

The comparison between the Entangled QFT and TEBD is instructive for quantum circuit design. The Entangled QFT’s  $n$  cross-dimensional phases outperform TEBD’s  $2n$  ring-topology phases because they connect degrees of freedom that are relevant to the 2D structure of images. The entanglement gates enable the basis to represent frequency components that mix row and column information, which no separable architecture can express regardless of parameter count. This observation suggests that for variational quantum circuits, the connectivity pattern should be guided by the structure of the

target problem rather than chosen for computational convenience.

While our circuits are evaluated classically as tensor networks, the parametric QFT structure maps directly to quantum hardware: implementing the Entangled QFT requires only  $n$  additional CPHASE gates beyond the standard QFT circuit. The tensor network contraction framework used in this work shares the same computational structure as quantum circuit simulation [6, 22], and can exploit the same software and hardware advances, including GPU acceleration and optimized contraction orders.

Our current benchmarks compare against the classical FFT; comparisons with the DCT, wavelets, and learned dictionaries would provide a more complete picture. Ablation studies isolating the contribution of Riemannian vs. projected Euclidean optimization, and evaluation of the MERA-inspired basis on power-of-two image patches, are natural next steps. A Julia implementation of the framework is available as an open-source library.

## 7 Conclusion

We have presented a framework for learning parametric quantum circuit bases for image compression, optimized on Riemannian manifolds of unitary and phase gates. By systematically comparing four circuit topologies, we find that circuit connectivity topology is a more important design variable than parameter count or circuit depth: the Entangled QFT, with  $O(\log N)$  additional cross-dimensional gates, consistently outperforms topologies with comparable or greater parameter counts, while TEBD’s local ring connectivity degrades below the fixed FFT baseline.

The framework unifies tensor network contraction, Riemannian optimization, and differentiable programming into a single pipeline. New circuit topologies can be defined by specifying a connectivity pattern; the manifold optimization, batched einsum evaluation, and differentiable truncation machinery apply unchanged. The approach adds another example along the fruitful line of research combining tensor networks, automatic differentiation, and quantum circuit structures for computational applications.

## Acknowledgments

## References

- [1] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [2] Nasir Ahmed, T Natarajan, and Kamisetty R Rao. Discrete cosine transform. *IEEE Transactions on Computers*, 100(1):90–93, 1974.
- [3] Guifré Vidal. Efficient simulation of one-dimensional quantum many-body systems. *Physical Review Letters*, 93(4):040502, 2004.
- [4] Guifré Vidal. Entanglement renormalization. *Physical Review Letters*, 99(22):220405, 2007.
- [5] Gary Bécigneul and Octavian-Eugen Ganea. Riemannian adaptive optimization methods. In *International Conference on Learning Representations*, 2019.
- [6] Xiu-Zhe Luo, Jin-Guo Liu, Pan Zhang, and Lei Wang. Yao.jl: Extensible, efficient framework for quantum algorithm design. *Quantum*, 4:341, 2020.
- [7] Martin Roa-Villescas, Xuanzhao Gao, Sander Stuijk, Henk Corporaal, and Jin-Guo Liu. Probabilistic inference in the era of tensor networks and differential programming. *Physical Review Research*, 6:033261, 2024.
- [8] Edwin Stoudenmire and David J Schwab. Supervised learning with tensor networks. *Advances in Neural Information Processing Systems*, 29, 2016.
- [9] Zhao-Yu Han, Jun Wang, Heng Fan, Lei Wang, and Pan Zhang. Unsupervised generative modeling using matrix product states. *Physical Review X*, 8(3):031012, 2018.
- [10] Ivan V Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.
- [11] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and

- Patrick J Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021.
- [12] Jarrod R McClean, Jonathan Romero, Ryan Babbush, and Alán Aspuru-Guzik. The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics*, 18(2):023023, 2016.
- [13] Maria Schuld and Nathan Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019.
- [14] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019.
- [15] Navin Khaneja, Timo Reiss, Thomas Schulte-Herbrüggen, and Steffen J Glaser. Optimal control of coupled spin dynamics: design of NMR pulse sequences by gradient ascent algorithms. *Journal of Magnetic Resonance*, 172(2):296–305, 2005.
- [16] P-A Absil, Robert Mahony, and Rodolphe Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2008.
- [17] Jin-Guo Liu, Lei Wang, and Pan Zhang. Tropical tensor network for ground states of spin glasses. *Physical Review Letters*, 126(9):090506, 2021.
- [18] Jin-Guo Liu, Xun Gao, Madelyn Cain, Mikhail D Lukin, and Sheng-Tao Wang. Computing solution space properties of combinatorial optimization problems via generic tensor networks. *SIAM Journal on Scientific Computing*, 45(3):A1239–A1270, 2023.
- [19] Google. Quick, draw! dataset. <https://quickdraw.withgoogle.com/data>, 2017.
- [20] Eirikur Agustsson and Radu Timofte. NTIRE 2017 challenge on single image super-resolution: Dataset and study. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 126–135, 2017.
- [21] David L Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.
- [22] Igor L Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.

## A Riemannian optimization details

### A.1 Cayley retraction derivation

For a unitary matrix  $U \in U(2)$  and tangent vector  $\xi = US$  where  $S^\dagger = -S$ , the Cayley retraction maps:

$$W = \xi U^\dagger = USU^\dagger, \quad (13)$$

$$W \leftarrow \frac{W - W^\dagger}{2} \quad (\text{project to } \mathfrak{u}(2)), \quad (14)$$

$$R_U(\alpha\xi) = \left(I - \frac{\alpha}{2}W\right)^{-1} \left(I + \frac{\alpha}{2}W\right) U. \quad (15)$$

Since  $W$  is skew-Hermitian,  $(I - \frac{\alpha}{2}W)^{-1}(I + \frac{\alpha}{2}W)$  is unitary for any  $\alpha \in \mathbb{R}$ , as can be verified by checking that the product with its adjoint equals the identity.

For  $2 \times 2$  matrices, the inverse admits a closed form via the adjugate:

$$(I - \frac{\alpha}{2}W)^{-1} = \frac{1}{\det(I - \frac{\alpha}{2}W)} \text{adj}(I - \frac{\alpha}{2}W). \quad (16)$$

### A.2 Parallel transport for Riemannian Adam

For  $U(2)$ , the projection-based transport from  $U_t$  to  $U_{t+1}$  is:

$$\Gamma_{U_t \rightarrow U_{t+1}}(m_t) = U_{t+1} \cdot \text{skew}(U_{t+1}^\dagger m_t). \quad (17)$$

For  $U(1)^4$ , the transport is the tangent projection at the new point:

$$\Gamma_{\mathbf{z}_t \rightarrow \mathbf{z}_{t+1}}(m_t) = i \cdot \text{Im}(\bar{\mathbf{z}}_{t+1} \circ m_t) \circ \mathbf{z}_{t+1}. \quad (18)$$

Both are exact to first order in the step size.

## B Detailed benchmark configurations



Table 5: Hyperparameters used in the benchmark experiments.

| Parameter                        | Value              |
|----------------------------------|--------------------|
| Optimizer                        | RiemannianGD       |
| Initial learning rate $\alpha_0$ | 0.01               |
| Armijo constant $c$              | $10^{-4}$          |
| Backtracking factor $\tau$       | 0.5                |
| Max backtracking steps           | 10                 |
| Loss function                    | MSE ( $k = 10\%$ ) |
| Validation split                 | 20%                |
| Early stopping patience          | 2 epochs           |
| Random seed                      | 42                 |
| Epochs (Quick Draw)              | 5                  |
| Training images (Quick Draw)     | 20                 |
| Epochs (DIV2K)                   | 5                  |
| Training images (DIV2K)          | 20                 |